19. ROS 2 Launch File Configuration

1. Introduction to Launch

Up to now, every time we run a ROS node, we need to open a new terminal and run a command. A robot system has many nodes, and starting them this way every time is troublesome. Is there a way to start all the nodes at once? The answer is of course yes: the launch file, which is a script for starting and configuring multiple nodes in the ROS system.

In ROS 2, launch is used for functions such as starting multiple nodes and configuring program runtime parameters. ROS 2 launch files come in xml, yaml, and python formats. This lesson takes a python-format launch file as an example. Compared with the other two formats, the python format is more flexible:

- python has numerous function libraries that can be used in launch files;
- ROS 2 general launch features and specific launch features are written in Python, so launch features that XML and YAML may not expose can be accessed;

When writing ROS 2 launch files in the python language, the most important thing is to abstract each node, file, script, and so on into an action, and start them with a unified interface.

References:

- Launch system design document: ROS 2 launch system design document
- Official launch API documentation: launch API documentation (updated with the official release)
- Preparation: create a package to store the program files

```
ros2 pkg create learn_launch --build-type ament_python
```

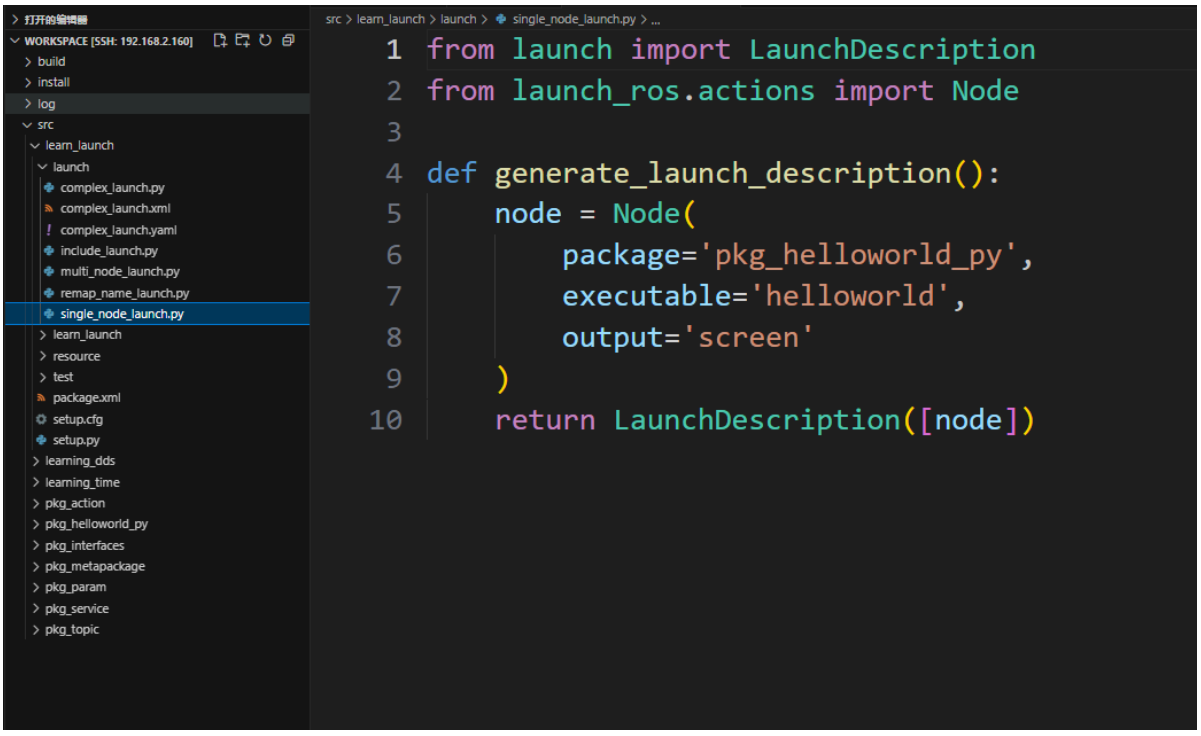
2. Writing the Launch for a Single Node

2.1. Creating a Launch File

Create a new launch folder under the package, then create a new 【single_node_launch.py】 file inside the launch folder, and copy the following content into the file:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    node = Node(
        package='pkg_helloworld_py',
        executable='helloworld',
        output='screen'
    )
    return LaunchDescription([node])
```

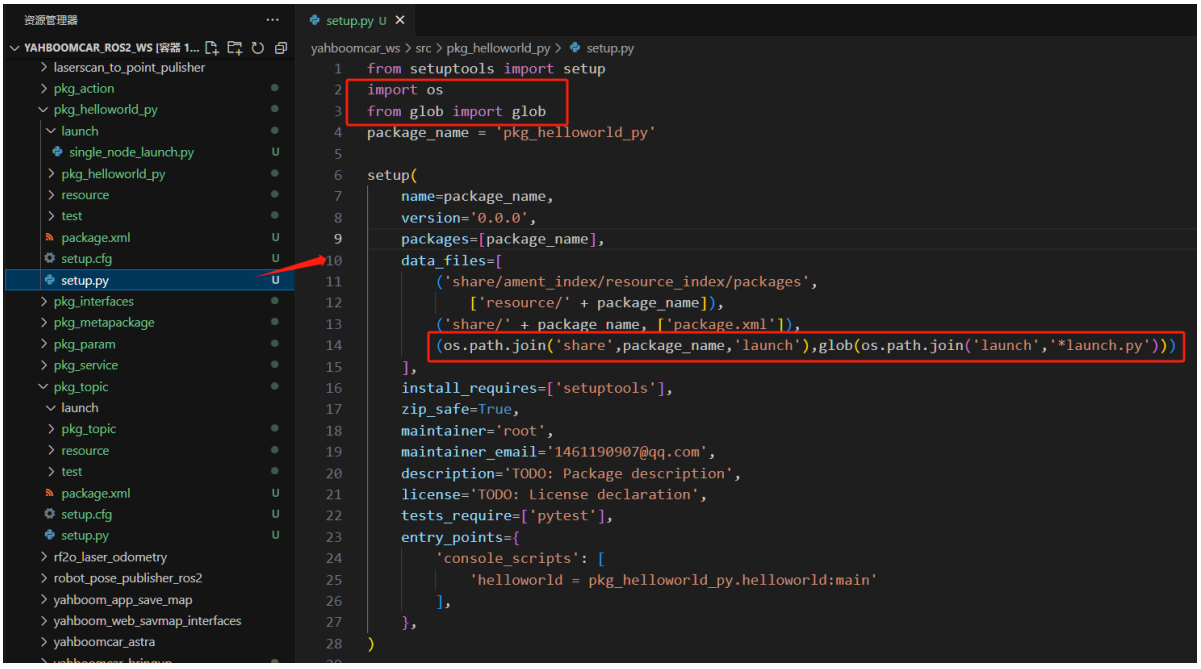


2.2. Configuring the setup.py File

Launch files are usually named LaunchName_launch.py, where LaunchName is user-defined and the `_launch.py` suffix is fixed. You need to modify the setup.py file under the package. The modification is to add the files under the launch path, so that an executable .py file is generated during compilation,

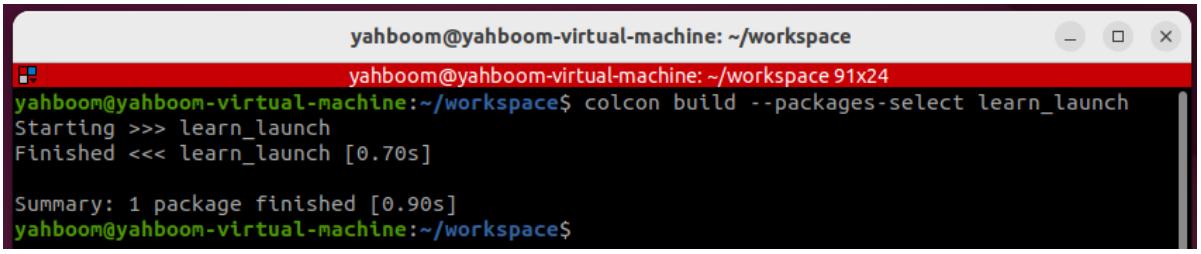
```
#1. Import the related header files
import os
from glob import glob

#2. In the data_files list, add the launch path and the launch.py file under the path
(os.path.join('share',package_name,'launch'),glob(os.path.join('launch','*launch.py')))
```



2.3. Building the Package

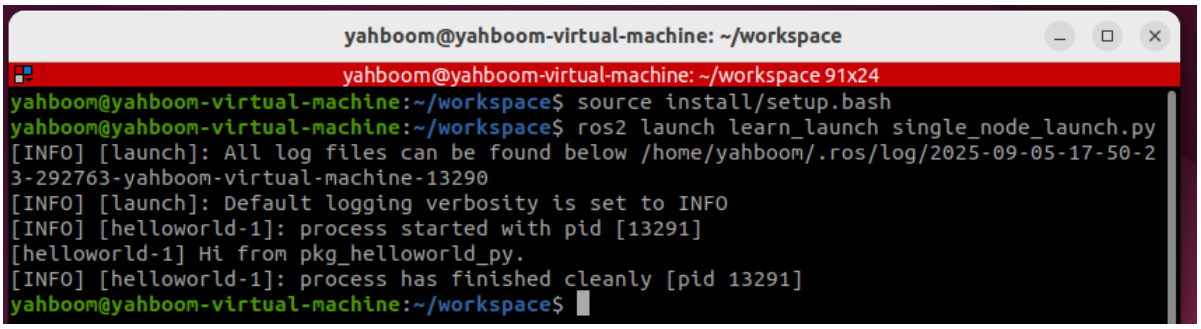
```
colcon build --packages-select learn_launch
```



2.4. Running the Program

- Refresh the environment variables, then run the launch file

```
ros2 launch learn_launch single_node_launch.py
```



2.5. Source Code Analysis

1. Import the related libraries

```
from launch import LaunchDescription
from launch_ros.actions import Node
```

2. Define a function generate_launch_description, and return a launch_description

```
def generate_launch_description():
    node = Node(
        package='pkg_helloworld_py',
        executable='helloworld',
    )
    return LaunchDescription([node])
```

A variable node is defined as the return value of starting a node. The Node function is called with two important parameters, package and executable.

- package: indicates the package, representing the name of the package.
- executable: indicates the program to execute, the name of the executable program.

Finally, call the LaunchDescription function, pass in the node parameter, and execute and return.

```
return LaunchDescription([node])
```

3. Writing the Launch for Multiple Nodes

3.1. Creating a Launch File

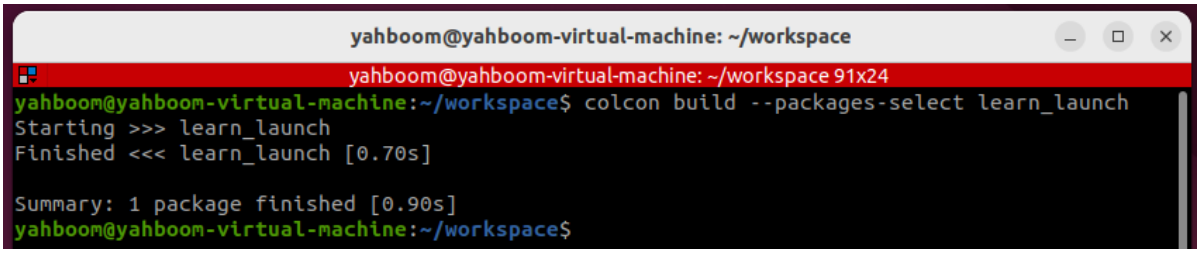
Create a new 【multi_node_launch.py】 file and add the following content:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    publisher_node = Node(
        package='pkg_topic',
        executable='publisher_demo',
        output='screen'
    )
    subscriber_node = Node(
        package='pkg_topic',
        executable='subscriber_demo',
        output='screen'
    )
    return LaunchDescription([
        publisher_node,
        subscriber_node
    ])
```

3.2. Building the Package

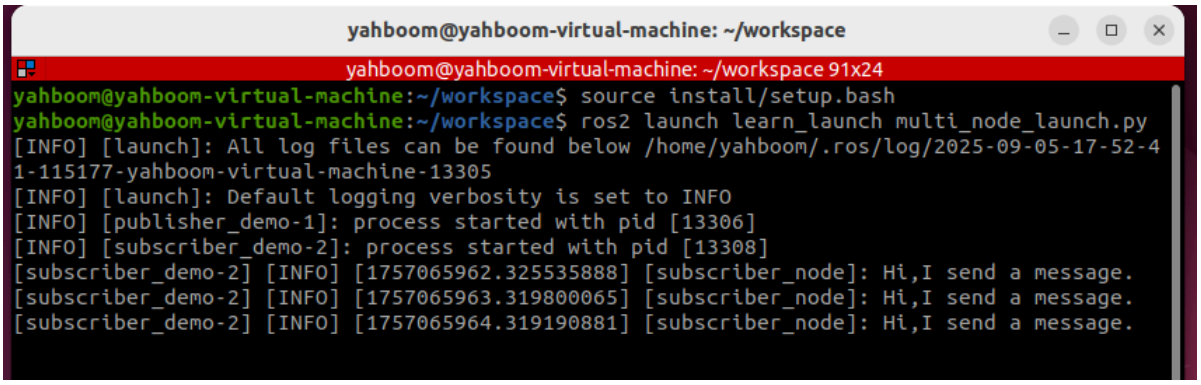
```
colcon build --packages-select learn_launch
```



3.3. Running the Program

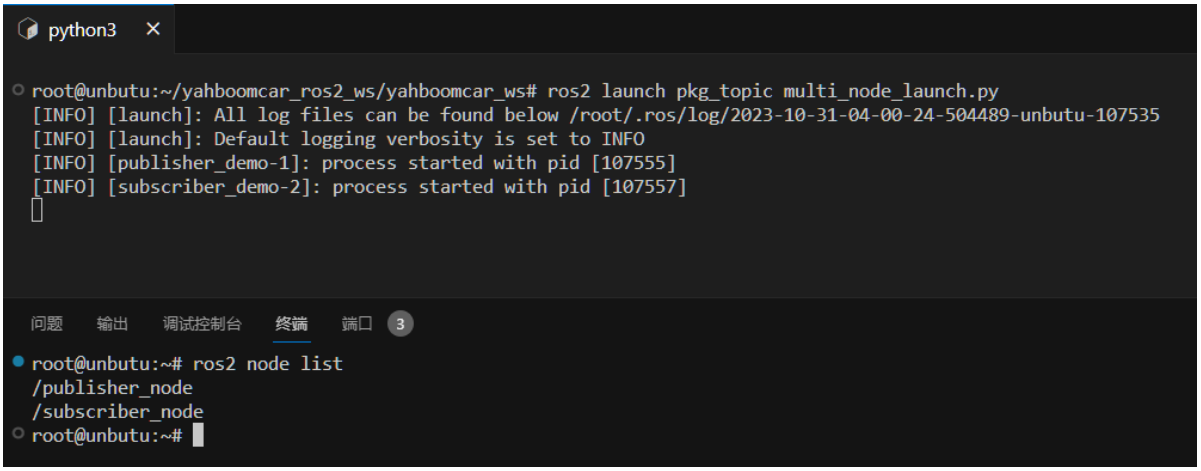
- Refresh the environment variables, then run the launch file

```
ros2 launch learn_launch multi_node_launch.py
```



If the terminal does not print anything, we can check which nodes are started to verify whether the startup succeeded. Enter in the terminal:

```
ros2 node list
```



3.4. Source Code Analysis

Roughly the same as simple_node_launch.py, except that there is one more node.

4. Topic Remapping Case

4.1. Creating a Launch File

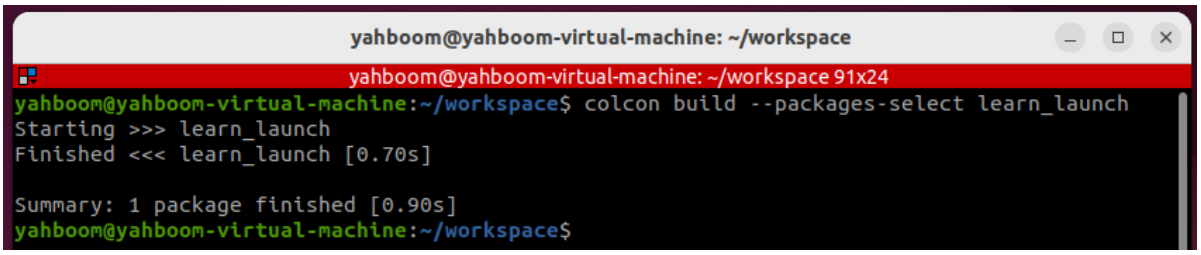
Create a new 【remap_name_launch.py】 file in the same directory as multi_node_launch.py and add the following content:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    publisher_node = Node(
        package='pkg_topic',
        executable='publisher_demo',
        output='screen',
        remappings=[("/topic_demo", "/topic_update")]
    )
    return LaunchDescription([
        publisher_node
    ])
```

4.2. Building the Package

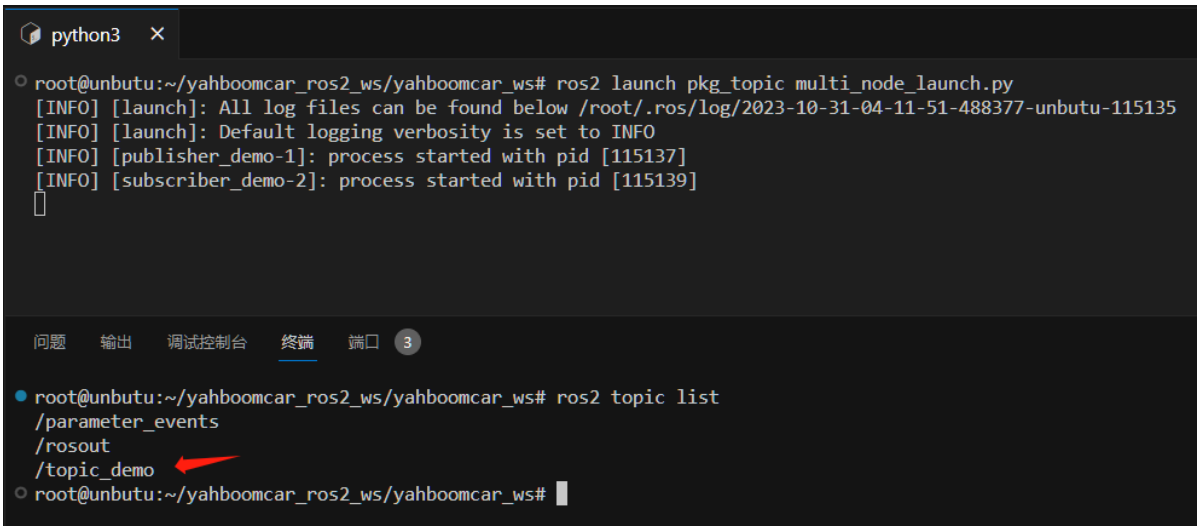
```
colcon build --packages-select learn_launch
```



4.3. Running the Program

We first look at what topics the publisher_demo node publishes before the topic is remapped:

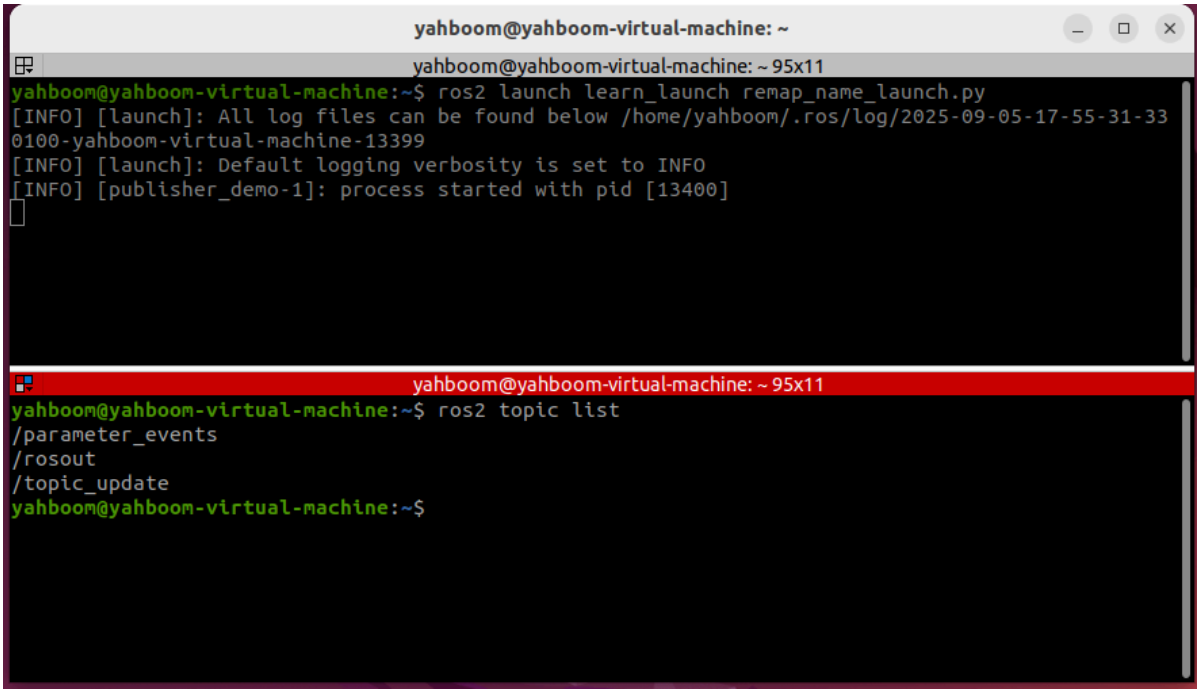
```
ros2 launch learn_launch multi_node_launch.py
ros2 topic list
```



The topic here is 【/topic_demo】

- Refresh the environment variables again and run the program after the topic remapping to see the change:

```
ros2 launch learn_launch remap_name_launch.py
ros2 topic list
```



As can be seen from the figure above, the topic name has been remapped to 【/topic_update】

4.4. Source Code Analysis

The following part is mainly added:

```
remappings=[("/topic_demo", "/topic_update")]
```

Here the original /topic_demo topic is remapped to /topic_update

5. Case of Nesting a Launch File to Start Another Launch File

5.1. Creating a Launch File

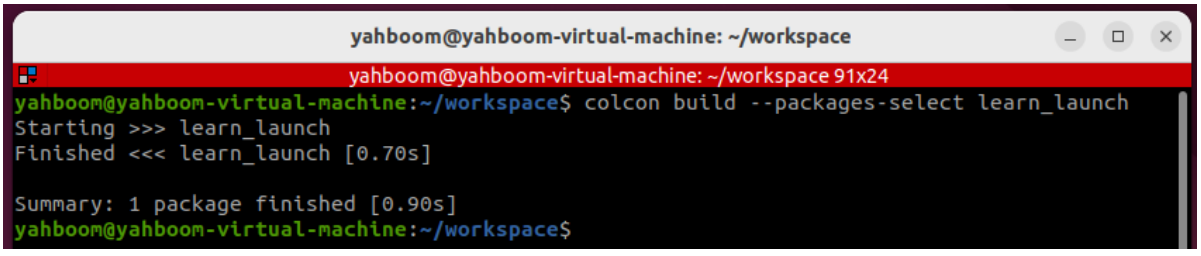
Create a new 【include_launch.py】 file in the same directory as multi_node_launch.py and add the following content:

```
from launch import LaunchDescription
import os
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    hello_launch = IncludeLaunchDescription(PythonLaunchDescriptionSource(
        os.path.join(
            get_package_share_directory('learn_launch'),
            'launch',
            'multi_node_launch.py'
        )),
    )
    return LaunchDescription([
        hello_launch
    ])
```

5.2. Building the Package

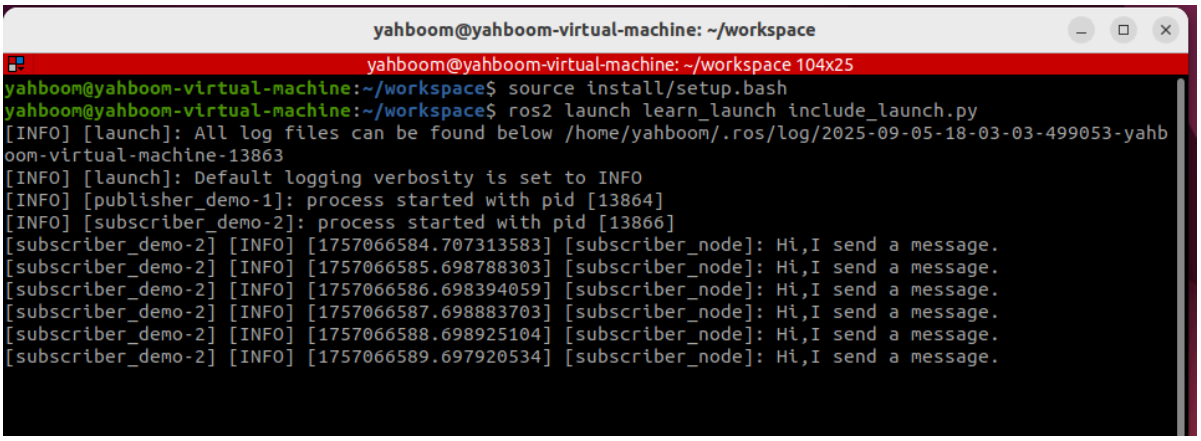
```
colcon build --packages-select learn_launch
```



5.3. Running the Program

- Refresh the environment variables and run the launch file

```
ros2 launch learn_launch include_launch.py
```



5.4. Source Code Analysis

- To nest and start a python-format launch file, you need to use the two classes `IncludeLaunchDescription` and `PythonLaunchDescriptionSource` of the launch system.
- `get_package_share_directory('learn_launch')`: obtains the share directory of the package, where `learn_launch` is the package name;
- `os.path.join(..., 'launch', 'multi_node_launch.py')`: uses relative path segments to join the included launch file. A `/` cannot be added before the file name, otherwise it will be treated as the file system root path.

6. Comprehensive Launch File Example

This case mainly shows how to write a complex launch file; the functionality of the program can be ignored.

6.1. Creating a Launch File

Create a new 【complex_launch.py】 file in the same directory as multi_node_launch.py and add the following content:

```
import os
from ament_index_python import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.actions import IncludeLaunchDescription
from launch.actions import GroupAction
from launch.launch_description_sources import AnyLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch.substitutions import TextSubstitution
from launch_ros.actions import Node
from launch_ros.actions import PushRosNamespace


def generate_launch_description():

    # args that can be set from the command line or a default will be used
    background_r_launch_arg = DeclareLaunchArgument(
        "background_r", default_value=TextSubstitution(text="0")
    )
    background_g_launch_arg = DeclareLaunchArgument(
        "background_g", default_value=TextSubstitution(text="255")
```

```
    )
    background_b_launch_arg = DeclareLaunchArgument(
        "background_b", default_value=TextSubstitution(text="0")
    )
    chatter_ns_launch_arg = DeclareLaunchArgument(
        "chatter_ns", default_value=TextSubstitution(text="my/chatter/ns")
    )

    # include another launch file
    launch_include = IncludeLaunchDescription(
        AnyLaunchDescriptionSource(
            os.path.join(
                get_package_share_directory('demo_nodes_cpp'),
                'launch/topics/talker_listener_launch.xml'))
    )
    # include another launch file in the chatter_ns namespace
    launch_include_with_namespace = GroupAction(
        actions=[
            # push-ros-namespace to set namespace of included nodes
            PushRosNamespace(LaunchConfiguration('chatter_ns')),
            IncludeLaunchDescription(
                AnyLaunchDescriptionSource(
                    os.path.join(
                        get_package_share_directory('demo_nodes_cpp'),
                        'launch/topics/talker_listener_launch.xml'))
            ),
        ]
    )

    # start a turtlesim_node in the turtlesim1 namespace
    turtlesim_node = Node(
            package='turtlesim',
            namespace='turtlesim1',
            executable='turtlesim_node',
            name='sim'
        )

    # start another turtlesim_node in the turtlesim2 namespace
    # and use args to set parameters
    turtlesim_node_with_parameters = Node(
            package='turtlesim',
            namespace='turtlesim2',
            executable='turtlesim_node',
            name='sim',
            parameters=[{
                "background_r": LaunchConfiguration('background_r'),
                "background_g": LaunchConfiguration('background_g'),
                "background_b": LaunchConfiguration('background_b'),
            }]
        )

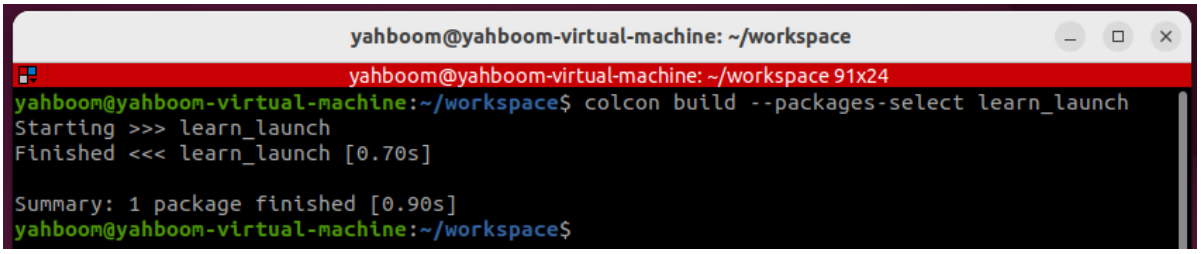
    # perform remap so both turtles listen to the same command topic
    forward_turtlesim_commands_to_second_turtlesim_node = Node(
            package='turtlesim',
            executable='mimic',
            name='mimic',
```

```
            remappings=[
                ('/input/pose', '/turtlesim1/turtle1/pose'),
                ('/output/cmd_vel', '/turtlesim2/turtle1/cmd_vel'),
            ]
        )

    return LaunchDescription([
        background_r_launch_arg,
        background_g_launch_arg,
        background_b_launch_arg,
        chatter_ns_launch_arg,
        launch_include,
        launch_include_with_namespace,
        turtlesim_node,
        turtlesim_node_with_parameters,
        forward_turtlesim_commands_to_second_turtlesim_node,
    ])
```

6.2. Building the Workspace

```
colcon build --packages-select learn_launch
```

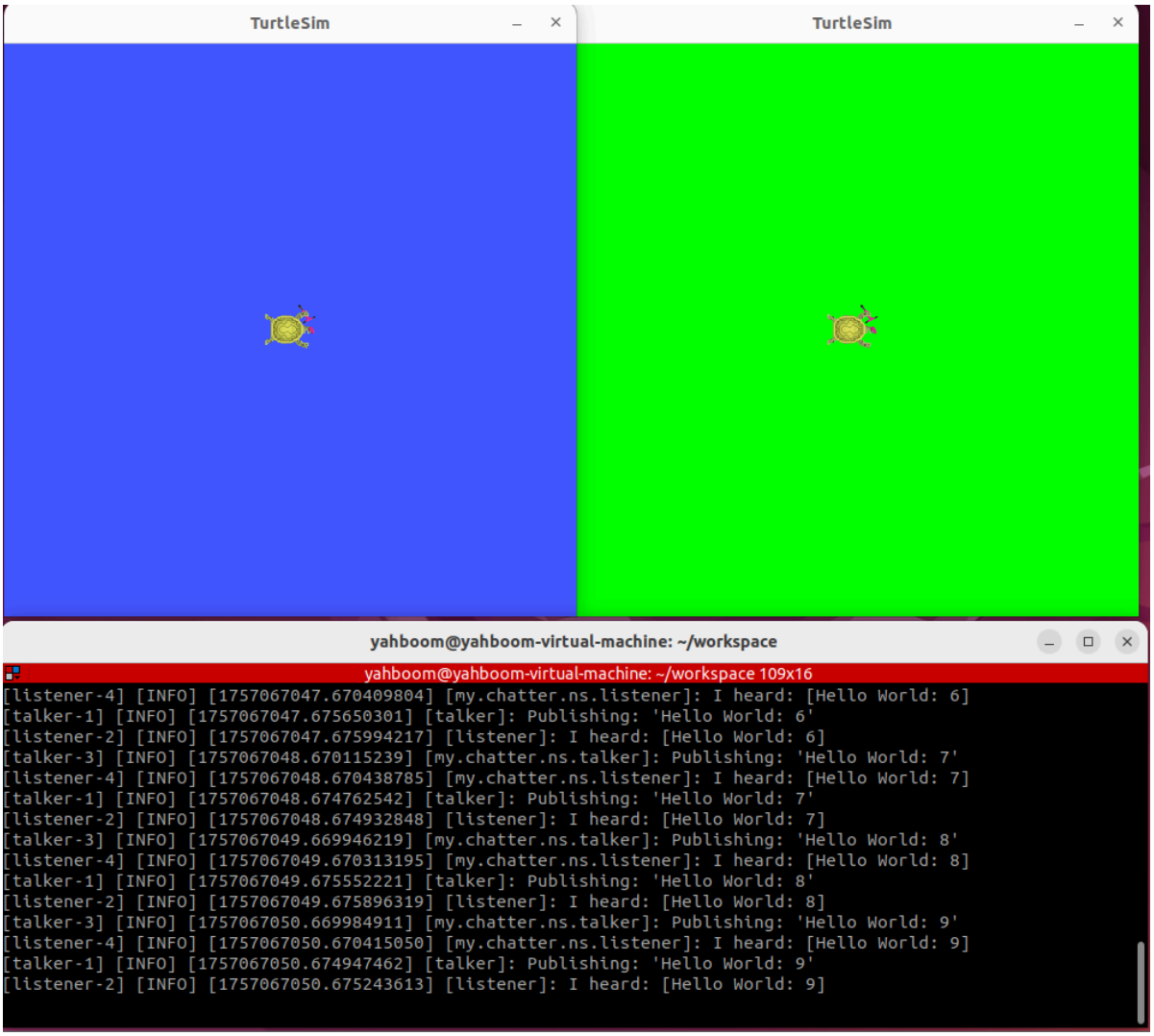


6.3. Running the Program

- Refresh the environment variables in the terminal and run the launch file

```
ros2 launch learn_launch complex_launch.py
```

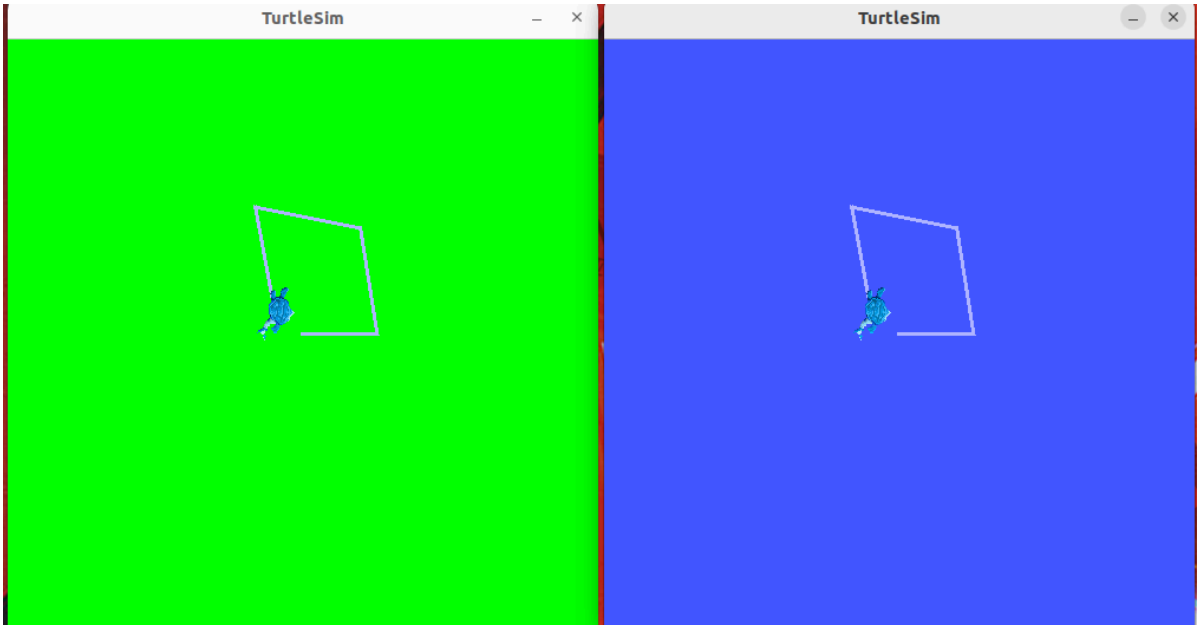
Two small turtles will be displayed on the host machine's vnc.



- Start the keyboard control node and add the namespace (because we added the namespace when starting the node in the launch file)

```
ros2 run turtlesim turtle_teleop_key --ros-args -r __ns:=/turtlesim1
```

- Use the up, down, left, and right keys to control the movement of turtle 1; turtle 2 will completely imitate the behavior of turtle 1



6.4. Program Description

The program mainly starts:

1. The talker_listener node of demo_nodes_cpp,
2. The talker_listener node with a namespace
3. Turtle 1 with turtlesim1 as its namespace
4. Turtle 2 with turtlesim2 as its namespace
5. Remapping is performed so that both turtles can hear the same command topic

7. XML Implementation

7.1. Creating a Launch File

Create a new 【complex_launch.xml】 file in the same directory as complex_launch.py and add the following content:

```
<launch>

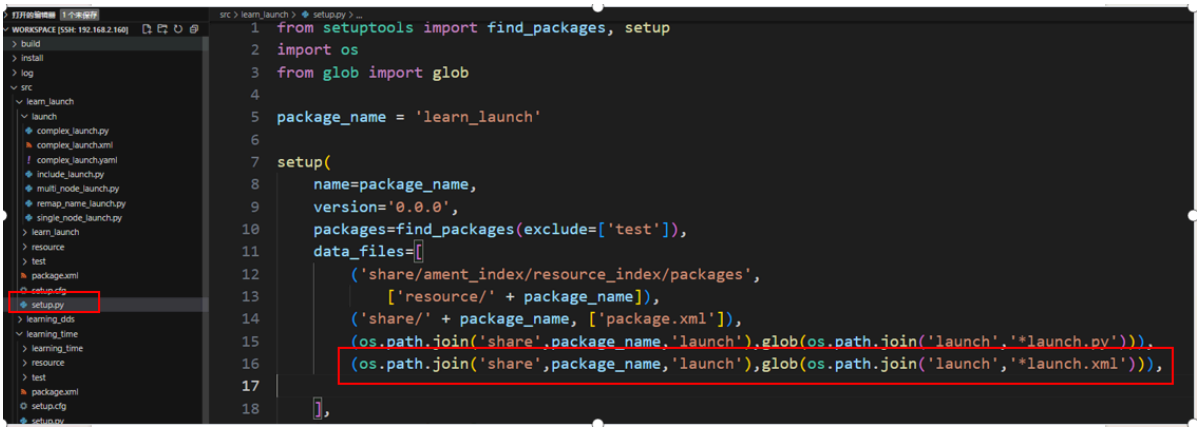
    <!-- args that can be set from the command line or a default will be used -->
    <arg name="background_r" default="0"/>
    <arg name="background_g" default="255"/>
    <arg name="background_b" default="0"/>
    <arg name="chatter_ns" default="my/chatter/ns"/>

    <!-- include another launch file -->
    <include file="$(find-pkg-share
demo_nodes_cpp)/launch/topics/talker_listener_launch.xml"/>
    <!-- include another launch file in the chatter_ns namespace-->
    <group>
      <!-- push-ros-namespace to set namespace of included nodes -->
      <push-ros-namespace namespace="$(var chatter_ns)"/>
      <include file="$(find-pkg-share
demo_nodes_cpp)/launch/topics/talker_listener_launch.xml"/>
    </group>

    <!-- start a turtlesim_node in the turtlesim1 namespace -->
    <node pkg="turtlesim" exec="turtlesim_node" name="sim"
namespace="turtlesim1"/>
    <!-- start another turtlesim_node in the turtlesim2 namespace
        and use args to set parameters -->
    <node pkg="turtlesim" exec="turtlesim_node" name="sim"
namespace="turtlesim2">
      <param name="background_r" value="$(var background_r)"/>
      <param name="background_g" value="$(var background_g)"/>
      <param name="background_b" value="$(var background_b)"/>
    </node>
    <!-- perform remap so both turtles listen to the same command topic -->
    <node pkg="turtlesim" exec="mimic" name="mimic">
      <remap from="/input/pose" to="/turtlesim1/turtle1/pose"/>
      <remap from="/output/cmd_vel" to="/turtlesim2/turtle1/cmd_vel"/>
    </node>
</launch>
```

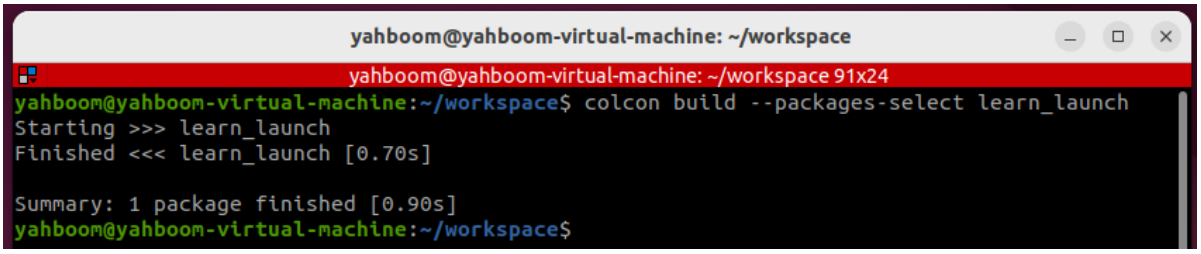
7.2. setup.py File Configuration

- You need to configure the build file, so that our .xml-format launch file is copied to the install directory during compilation, and only then can the ros system find our file



7.3. Building the Package

```
colcon build --packages-select learn_launch
```

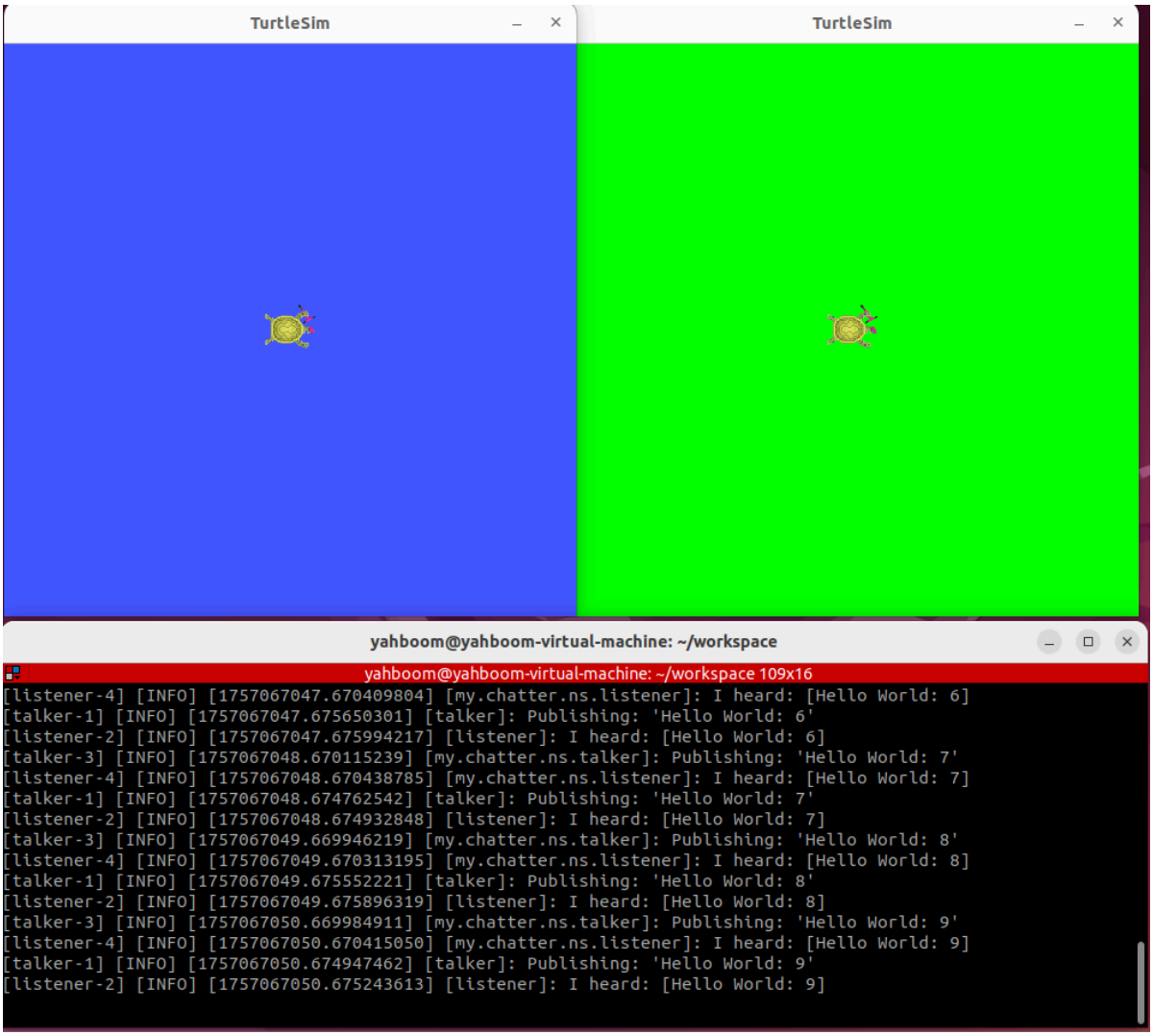


7.4. Running the Program

Enter in the terminal:

```
ros2 launch learn_launch complex_launch.xml
```

- As expected, two small turtles will appear, and the terminal will print log information



- Start the keyboard control node and add a namespace

```
ros2 run turtlesim turtle_teleop_key --ros-args -r __ns:=/turtlesim1
```

Use the keyboard control to start turtle 1 running; turtle 2 will completely imitate the behavior of turtle 1

8. YAML Implementation

8.1. Creating a Launch File

Create a new 【complex_launch.yaml】 file in the same directory as complex_launch.py and add the following content:

```
launch:

# args that can be set from the command line or a default will be used
- arg:
    name: "background_r"
    default: "0"
- arg:
    name: "background_g"
    default: "255"
- arg:
    name: "background_b"
```

```
    default: "0"
- arg:
    name: "chatter_ns"
    default: "my/chatter/ns"


# include another launch file
- include:
    file: "$(find-pkg-share
demo_nodes_cpp)/launch/topics/talker_listener_launch.xml"

# include another launch file in the chatter_ns namespace
- group:
    - push-ros-namespace:
        namespace: "$(var chatter_ns)"
    - include:
        file: "$(find-pkg-share
demo_nodes_cpp)/launch/topics/talker_listener_launch.xml"

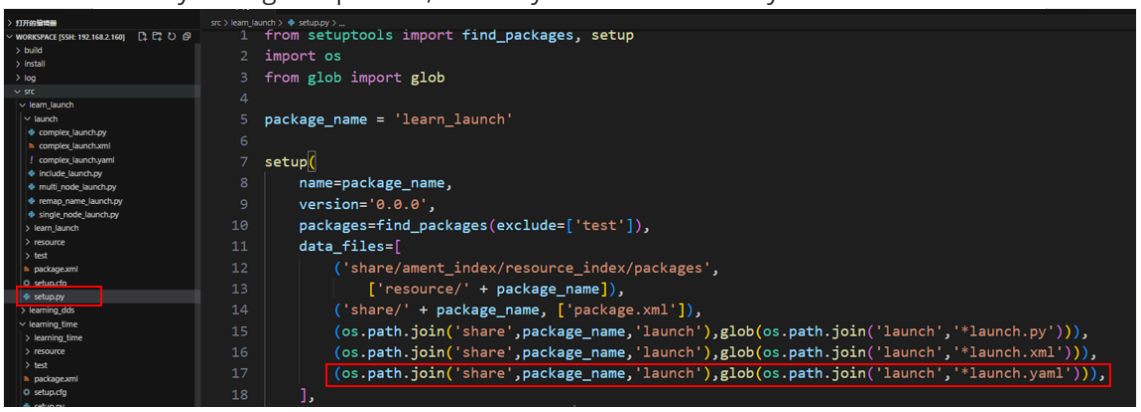
# start a turtlesim_node in the turtlesim1 namespace
- node:
    pkg: "turtlesim"
    exec: "turtlesim_node"
    name: "sim"
    namespace: "turtlesim1"

# start another turtlesim_node in the turtlesim2 namespace and use args to set
parameters
- node:
    pkg: "turtlesim"
    exec: "turtlesim_node"
    name: "sim"
    namespace: "turtlesim2"
    param:
    -
      name: "background_r"
      value: "$(var background_r)"
    -
      name: "background_g"
      value: "$(var background_g)"
    -
      name: "background_b"
      value: "$(var background_b)"

# perform remap so both turtles listen to the same command topic
- node:
    pkg: "turtlesim"
    exec: "mimic"
    name: "mimic"
    remap:
    -
        from: "/input/pose"
        to: "/turtlesim1/turtle1/pose"
    -
        from: "/output/cmd_vel"
        to: "/turtlesim2/turtle1/cmd_vel"
```

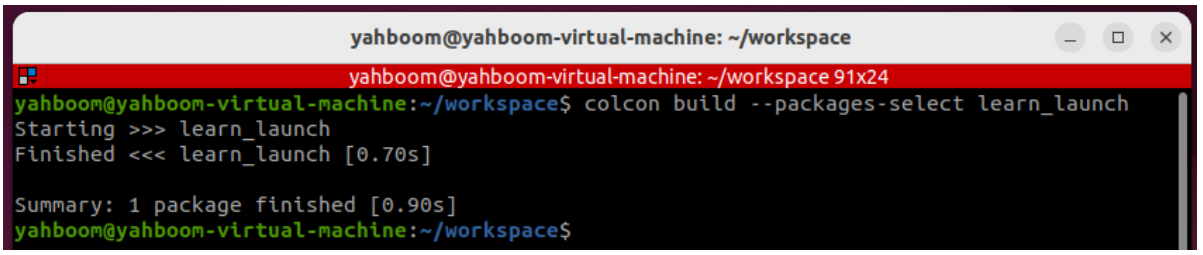
8.2. Configuration

- You need to configure the build file, so that our .yaml-format launch file is copied to the install directory during compilation, and only then can the ros system find our file



8.3. Building the Package

```
colcon build --packages-select learn_launch
```

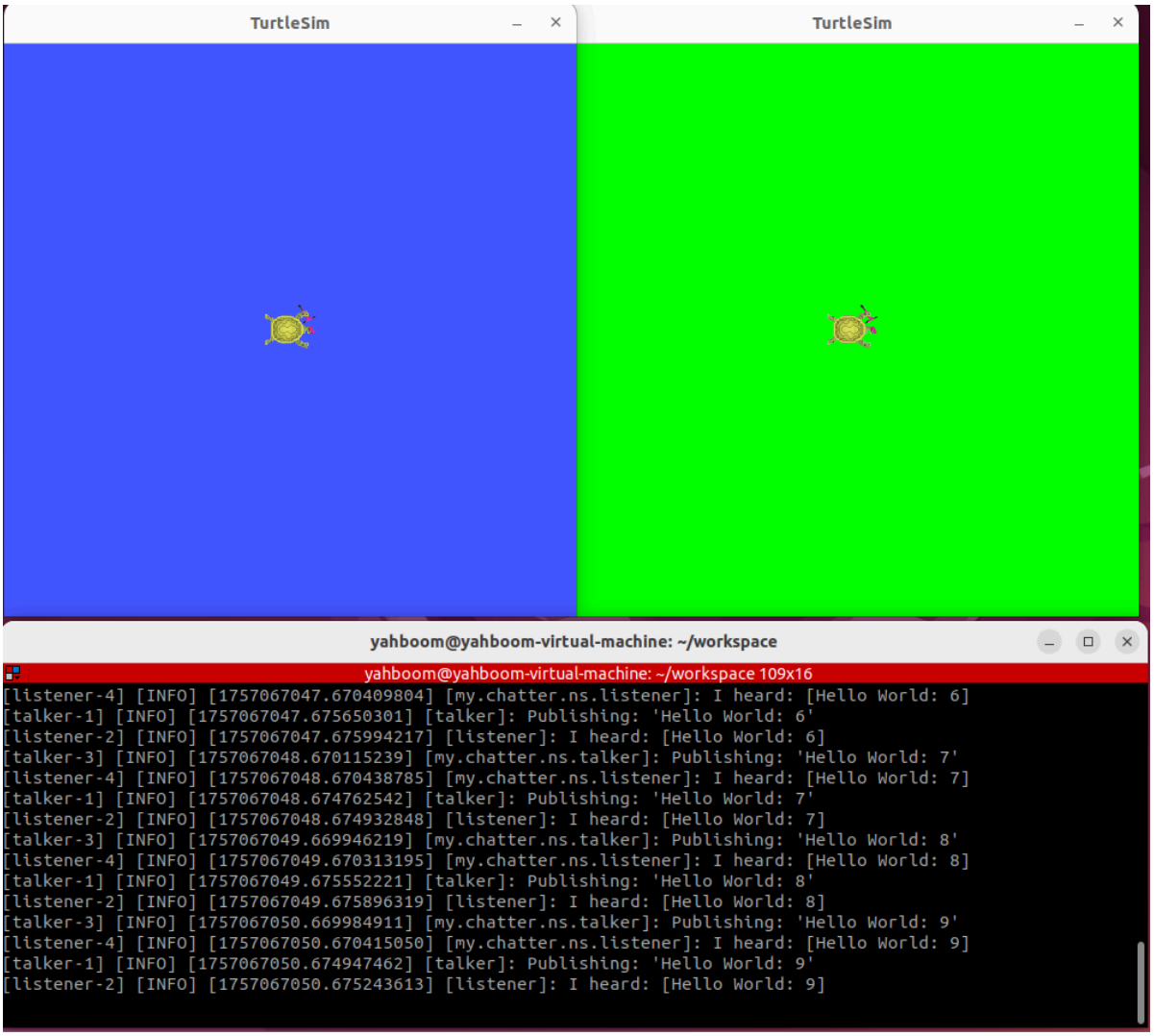


8.4. Running the Program

- Refresh the environment variables and then run

```
ros2 launch learn_launch complex_launch.yaml
```

- As expected, two small turtles will appear, and the terminal will print log information



- Start the keyboard control node and add a namespace

```
ros2 run turtlesim turtle_teleop_key --ros-args -r __ns:=/turtlesim1
```

Use the keyboard control to start turtle 1 running; turtle 2 will completely imitate the behavior of turtle 1